

Exact Execution Conditioning for Boolean Rule Infilling

Research supplement: algorithm, guarantees and comparison boundaries

Gabriel Jota Lizardo

October 8, 2026

Abstract

We extend a finite-original-token grammar posterior with execution profiles for Boolean JsonLogic rules. Conditional samples satisfy declared input-output examples while retaining the frozen dLLM product weights. The implementation uses the established covering-product transform. We give a tight scoped comparison: computing all union-profile masses needs 3^m products in a non-negative bilinear plan, whereas a signed plan needs exactly 2^m . We additionally condition only on counterexamples discovered during sampling. For m finite execution requirements, the entire stream rejects at most m rules and retains the exact conditional law. A non-prefix parity requirement exhibits an exponential representation gap against zero-rejection prefix exclusion. Lean checks the finite progress and prefix-cover obstructions, as well as the earlier rectangle bound. Further, positive auxiliary weights certify when active requirements imply every omitted example; this certificate survives arbitrary subsequent model reweightings and admitted support contractions. The inference exponent then depends on the certified core size rather than the number of supplied records. This is an incremental reference implementation and a delimited mathematical argument, not a claim of priority for transforms, program synthesis or exact inference. Independent human review and practical deployment evaluation remain pending.

1 Operation and antecedents

Given a fixed token canvas, finite original-token supports D_i , rational weights q_i , and m Boolean records, let $\mathcal{V}$ be the grammar-valid completions preserving committed positions. Every completion consumes exactly the declared slots with absent EOS. For a rule y , define its execution profile $h(y) \subseteq [m]$ by the records on which it returns true. We compute

$$Z_T = \sum_{y \in \mathcal{V}: h(y)=T} \prod_i q_i(y_i)$$

and sample tokens with this normalized conditional law when $Z_T > 0$. Rows truncated to a lexical domain retain their original normalization; exactness is on that domain. Zero mass is distinct from resource refusal.

The concrete language consists of Boolean fields, binary **and/or** and unary **!**, serialized as recursive JsonLogic objects. Matching the examples is a useful guarantee for completing a software-consumed filter or visibility condition. It is not correctness on unseen records. The reference admits 1–12 records in its profile backend and ASCII identifier fields. Its grammar is independent of examples and desired labels. An identifier trie distinguishes names such as **active** and **active_admin**, including token boundaries inside their common prefix. Records have Boolean values; dotted paths and general JsonLogic types are not admitted. An opt-in **!!** production passes through a Boolean value and preserves the original grammar by default. The adaptive stream can verify more than twelve records, but its active-profile backend still refuses a thirteenth active requirement. That refusal cannot establish infeasibility.

Tree automata for synthesis from examples, including Boolean formulas, are established: see Wang et al., Example 6.2 and Theorem 6.3 [5]. Exact weighted inference is likewise established,

and FactorDLM can represent deterministic semantic constraints [4]. The transform below is the classical covering product, not a new subset-convolution algorithm [1].

2 Inference and exact sampling

Lift each Boolean node of the unambiguous token-path forest to an array $W[S]$, $S \subseteq [m]$. A variable has its record-determined profile; punctuation is scalar. Alternatives sum arrays. NOT complements the indices. OR combines arrays by

$$(U \star V)[T] = \sum_{A \cup B = T} U[A]V[B].$$

AND applies the same product to complemented arrays and complements its output. With $\zeta U[S] = \sum_{A \subseteq S} U[A]$, compute $U \star V = \zeta^{-1}((\zeta U)(\zeta V))$ in $O(m2^m)$ ring work [1, Section 2.5, equations (15)–(17)]. Use exact integers scaled by one common denominator per token position; intermediate subtraction introduces no floating-point cancellation error. For sparse children, when the product of positive-support sizes is at most 2^m , the reference enumerates only those pairs; otherwise it uses transforms. The sampler uses the analogous sparse alternative. Both evaluate the same sums and retain the $O(m2^m)$ bound. This input-dependent dispatch is not itself a fixed non-negative bilinear plan.

Theorem 1 (Execution posterior). *The root array, divided by the product of row denominators, equals $(Z_T)_T$. For $Z_T > 0$, the recursive conditional sampler returns y with probability $\prod_i q_i(y_i)/Z_T$ on $\mathcal{V} \cap h^{-1}(T)$.*

Proof. Induct on increasing forest spans. Children occupy disjoint byte paths; combining their unique profiles implements the operator semantics. Sum over disjoint alternatives, using the original LL(1) unambiguous grammar. Aliases remain separate events because each original token closes on its own edge. Every full path has the same row denominator. For this restricted grammar, there are no unit/epsilon source productions: forced binarization and terminal proxies only distribute punctuation and pass single Boolean attributes. AND/OR have two Boolean children before binarization; NOT complements its single Boolean child. This is not a claim about arbitrary attributed-grammar normalization.

For OR conditioned on T , restrict V to subsets of T and compute $s(R) = \sum_{B \supseteq R, B \subseteq T} V[B]$ with a superset zeta transform. Draw $A \subseteq T$ with weight $U[A]s(T \setminus A)$, then B with weight $V[B]$ among $A \cup B = T$. Their joint probability is $U[A]V[B]/(U \star V)[T]$. AND uses complements; NOT and punctuation are deterministic in the profile. Choosing forest alternatives by their requested profile masses telescopes these conditional probabilities to the stated law. $\square$

3 Counterexamples replace repeated blind rejections

Let $G_i \subseteq \mathcal{V}$ be the rules satisfying execution requirement i , $G = \bigcap_{i=1}^m G_i$, and $w(y) = \prod_i q_i(y_i)$. Initially $A = \emptyset$. Sample exactly from w conditioned on $H_A = \mathcal{V} \cap \bigcap_{i \in A} G_i$. Execute the sampled rule on all requirements, stopping at the first violation. Return it only if it satisfies all requirements; otherwise add one violated index to A , recompute the profile posterior on those active records, and try again. Preserve A across returned samples. Neither the canvas nor the model probabilities change.

Theorem R 1 (Exact stream and a finite rejection budget). *Assume exact conditional sampling and finite evaluable requirements. If $Z_G = \sum_{y \in G} w(y) > 0$, every returned rule has law $w(y)/Z_G$, and any N returned rules are independent with this law. Over the entire stream, there are at most m rejected rules: producing N rules uses at most $N + m$ complete candidate draws. If $Z_G = 0$, after at most m rejections an active posterior has zero mass and proves zero mass on the full declared support. These guarantees assume sufficient resources for each required refinement.*

Proof. A conditional candidate satisfies every active requirement. Its violated index is therefore new. Each rejection increases $|A|$ by one; at $A = [m]$ the proposal is already conditioned on G . Also $G \subseteq H_A$, so a zero active mass certifies zero full mass. When $Z_G > 0$, every active mass is positive.

For the law, fix any active set A and earlier output history. Induct on the number of inactive requirements. Write $Z_A = \sum_{y \in H_A} w(y)$ and $F_A(y)$ for the probability of the next returned valid rule. The first draw either returns y or rejects some $x \in H_A \setminus G$ and adds its violated index $i(x)$. The induction hypothesis gives

$$F_A(y) = \frac{w(y)}{Z_A} + \sum_{x \in H_A \setminus G} \frac{w(x)}{Z_A} \frac{w(y)}{Z_G} = \frac{w(y)}{Z_A} + \left(1 - \frac{Z_G}{Z_A}\right) \frac{w(y)}{Z_G} = \frac{w(y)}{Z_G}.$$

The base case $H_A = G$ is immediate. The same law holds after every output history, even though active sets are random, proving independence. $\square$

Adaptive exactness is an established principle, including CARS [3, Section 3 and Appendix E]; learning requirements from counterexamples is established in example-guided synthesis [5]. The application-specific refinement differs from CARS’s prefix exclusions: it removes *all* rules violating a selected record, including rules with unrelated prefixes. It supplies the finite m bound without an exponentially enumerated perfect prefix oracle. This is a scoped algorithmic extension, not an invention of adaptive rejection.

Theorem R 2 (A prefix-exclusion representation gap). *Consider $n \geq 1$ independent unary-operator choices, each either $!$ or $!!$, wrapping a fixed Boolean variable, with fixed closing punctuation. All 2^n choices are syntactically valid original-token paths. Require one output Boolean value on one record. With strictly positive row weights, a prefix-exclusion proposal supported exactly on satisfying programs needs at least 2^{n-1} excluded prefixes. Execution-profile inference needs two profile values per Boolean forest cell, and the adaptive stream rejects at most one rule before all later draws satisfy the requirement.*

Proof. The output is the variable value toggled by the parity of $!$ choices. Exactly half the choices violate the target. Every proper prefix of the n choices extends to a satisfying program by choosing the last toggle; it cannot be excluded soundly. Thus each excluded prefix fixes all n choices and covers at most one of the 2^{n-1} invalid assignments. This applies even with a perfect prefix-validity oracle, independent of how the trie is optimized. For the profile representation, one record has two possible values; NOT exchanges them and identity preserves them. The fixed grammar/token DAG have $O(n)$ size, a CFG forest has at most $O(n^3)$ alternatives, and preparation and profile inference are polynomial rather than an enumeration of 2^n programs. One failed candidate activates the sole requirement. $\square$

The gap concerns a *zero-rejection representation*, not time to obtain one valid sample or a universal expected-time lower bound for CARS. A trie that remains small can keep rejecting; ordinary rejection for balanced parity uses only two trials on average. A two-state parity automaton, an adequate factor graph or another tree-automaton semantic solver shares the compactness advantage. This mathematical family diagnoses a representation limit; it is not a favorable performance benchmark. EPIC is a syntax/progress baseline and does not, by itself, implement this execution-conditioned sampling task.

If k records become active, the retained syntax forest is not recompiled. For a source grammar of size g , profile work across refinements is at most $O((g + S + F)k + F \sum_{j=1}^k j2^j) = O((g + S + F)k + Fk2^k)$; candidate sampling uses at most $N + k$ draws and $O((N + k)F \max(1, k2^k))$ arithmetic work. Verifying records costs at most $O(m(N + k)B)$, with B bounding one rule’s execution size. Memory includes the input records, $O(md)$ for d fields, and $O(F2^k)$ profile entries. Bit costs of the exact integers and syntax preparation remain additional as below.

There is no guarantee that $k \ll m$ or that this is faster than eager conditioning or enumeration on small supports. The exact proposal mass is an upper bound on Z_G ; it is not reported as full-specification mass.

Every CNF derivation has at most twice as many nodes as byte-path edges, and an acyclic path has fewer edges than graph vertices. Using this fact, the reference reserves a conservative whole-traversal work bound before conditional sampling, rather than timing out selected paths during a draw. Profile admission/refinement caps fail between complete candidate draws and make the session terminal. Returning only paths whose randomly selected traversals happened to fit a budget would require a separate bias analysis. External interruption and hard integer-operation preemption are not covered.

4 Certifying a reusable semantic core

An active set need not include every supplied record. Let $c_i(A) = |H_A \setminus G_i|$ count original-token paths violating an omitted requirement. Compute each $c_i(A)$ separately with profiles for $A \cup \{i\}$ and positive auxiliary weights: use $1/|D_j|$ for every token in slot j , including tokens with zero probability in the current model prediction. Multiplying the resulting rational mass by $\prod_j |D_j|$ gives an integer path count. Committed rows are singletons with probability one. This calculation does not replace the model weights used to generate samples.

Theorem R 3 (Structural core and arbitrary reweighting). *If $c_i(A) = 0$ for every omitted requirement, then $H_A = G$ on the declared finite support. Consequently conditioning any subsequent non-negative product weights on A gives exactly the full-specification mass and sampling law. This remains true after support contraction or adding fixed positions. Support expansion, releasing fixed positions, changing examples or changing grammar/tokenizer/EOS semantics requires a new certificate.*

Proof. $G \subseteq H_A$ always. Any $y \in H_A \setminus G$ violates some G_i , so contributes one to $c_i(A)$, contradicting zero. Positive auxiliary weights ensure no represented path is hidden merely because its current model weight vanishes. Thus the two indicator functions agree pointwise, independently of weights. Multiplying by any new path weight and summing proves mass equality; normalizing a positive mass proves law equality. The pointwise equality is preserved on subsets of the certified path domain. New paths are not covered. $\square$

The compiler greedily adds the omitted requirement with the greatest structural violation count, breaking ties by its declared index, until every count is zero. Each positive refinement removes at least one path and adds a fresh requirement. It finds a sufficient core, not necessarily a minimum one; an empty active language also certifies an impossible specification. An optional candidate core may be supplied, but is checked rather than assumed sound. Selecting examples, teaching dimension and set-cover formulations are established antecedents [2, Section 5]. The operation here preserves *every* consistent original-token completion and its changing product weights, rather than identifying just one desired program. This is a specialization and implementation contribution, not a claim to invent logical redundancy.

For a returned core of size k , certification uses at most $k + 1$ profile dimensions, even with $m > 12$ supplied records. It evaluates at most $m(k + 1)$ individual probes. Its arithmetic cost is $O(m(k + 1)(g + S + F) + Fm(k + 1)2^k)$, and peak profile space is $O(F2^{k+1})$, plus the $O(md)$ supplied data. After certification, each new weight query uses $O(g + S + F \max(1, k2^k))$ arithmetic work and $O(F2^k)$ profile space. Integer bit costs remain as below. For T predictions, include the one-time certification cost plus all T reweightings; compilation is not free. Compared with the retained dense all-record profile implementation, profile array lengths decrease from 2^m to 2^k per query. This is a conditional implementation advantage, not a lower bound on a competent single-target or sparse semantic algorithm. When k is large or few queries are requested, certification can cost more than the work it saves.

There are application families with genuinely distinct, redundant records. For monotone Boolean rules, matching all minimal positive and maximal negative records in the coordinatewise order implies matching every supplied record: each positive has a positive boundary record below it, and each negative has a negative boundary record above it. In particular, the specification $f(x) = x_a$ over all 2^d Boolean records has a two-record sufficient core: $(x_a = 1, x_{b \neq a} = 0) \mapsto 1$ and $(x_a = 0, x_{b \neq a} = 1) \mapsto 0$. Monotonicity sandwiches all other values. The syntax and probabilities need not specify the correct program: many redundant formulas, such as $a \wedge a$ and $a \vee a$, remain different valid token events. This observation motivates filters and monotone visibility rules; it is an elementary consequence of monotonicity, not a new learning theorem. The compiler checks implication even when the provided grammar/support also admits non-monotone programs, and adds requirements or refuses if necessary.

Lean checks the finite zero-violation implication, complete-predicate equality and equality of arbitrary natural-weight sums on restricted domains. Positive uniform CFG path counts, rational denominator clearing, normalization and the Python compiler remain written or independently enumerated obligations. Focused tests deliberately include initially zero-weight invalid tokens that later regain mass, support expansion beyond a narrower certified state, released commitments, contradictory specifications, aliases and independently enumerated alternative programs. These reject unsound probability-dependent constraint removal. One finite correctness corollary uses all 64 distinct records of six Boolean fields and a generic AND/OR canvas with 72 original-token programs. Two supplied boundary requirements are certified to imply all 64; exactly two programs remain, independently enumerated. The all-record reference refuses its twelve-record admission cap, whereas certification uses only three profile dimensions per probe and two per later query. This is an admitted-operation and correctness check, not a timed benchmark or a claim that other semantic algorithms cannot solve it.

5 A tight comparison for the required join

We compare the same complete profile operation on arbitrary dense arrays, not different decoding objectives or a deliberately poor factor encoding. A non-negative bilinear plan with r products has

$$C_T = \sum_{j=1}^r w_{jT} \left(\sum_A u_{jA} U_A \right) \left(\sum_B v_{jB} V_B \right), \quad w_{jT}, u_{jA}, v_{jB} \geq 0.$$

We count cross-input bilinear products; linear work and constant coefficients are additional costs. Allowing signed coefficients defines the signed class.

Theorem 2 (Bilinear product counts). *For all union-profile outputs, the minimum non-negative product count is 3^m . The minimum signed product count is 2^m . Complementing indices gives the same result for intersection-profile outputs.*

Proof. Non-negativity prevents cancellation of a wrong monomial. Each useful component therefore defines a rectangle $L_j \times R_j \times O_j$ on which $A \cup B = T$ holds. A fixed pair (A, B) has a unique output, so a useful rectangle cannot share different output profiles.

For each T , take the $2^{|T|}$ witness pairs $(A, T \setminus A)$, $A \subseteq T$. If a rectangle contains two distinct pairs for the same T , it contains both cross pairs. Their unions must also equal T , requiring $A' \subseteq A$ and $A \subseteq A'$. Thus each rectangle covers at most one witness. There are $\sum_T 2^{|T|} = 3^m$ witnesses, one of three possibilities per coordinate: neither side, left only, or right only.

The lower bound is attained by the products

$$U[A] \sum_{B \subseteq T, B \supseteq T \setminus A} V[B], \quad T \subseteq [m], \quad A \subseteq T.$$

A competent recursive implementation also attains it: split arrays on a bit, $C_0 = U_0 \star V_0$ and $C_1 = U_1 \star (V_0 + V_1) + U_0 \star V_1$. It uses 3^m products and $2(3^m - 2^m)$ additions.

The established zeta/product/inversion algorithm uses 2^m bilinear products and $3m2^{m-1}$ additions/subtractions. For optimality, the output polynomials are independent: the monomial $U[T]V[\emptyset]$ occurs only in C_T . A plan with r product polynomials spans at most an r -dimensional output space, hence $r \geq 2^m$. $\square$

The non-negative comparator is optimal in its declared class. The result does not lower-bound every factor encoding, a single target mass, adaptive sparse algorithms or all inference. A classical solver incorporating the same transform shares the advantage. No general superiority over EPIC or FactorDLM is implied. Rejection from the same syntax-conditioned law is also exact for this task and may be cheaper; its expected trial count is $1/\Pr(h(Y) = T \mid Y \in \mathcal{V})$ when that probability is positive.

6 Costs, implementation and evidence boundaries

Let F denote the explicit forest size and S the token support size. Preparation, including bounded normalization and token-DAG/forest construction, is additional. Evaluation uses $O(S + Fm2^m)$ arithmetic work and $O(F2^m)$ entries. Recomputed term contributions and conditional pair joins give an $O(Fm2^m)$ upper bound per sample. The reference also performs the existing scalar posterior evaluation for compatibility and mass-partition checks; its $O(F + S)$ cost is included. The method remains exponential in record count.

If span masses have at most L bits, transform intermediates have $O(L + m)$ bits. Each join therefore costs $O(m2^m(L + m) + 2^m M(L + m))$ bit work, where M is integer multiplication cost; input rationalization and output costs are separate. A safe L follows from the bit lengths of integer row weights and the product of row support sizes bounding token paths. Fewer products does not guarantee lower wall time on every input. Memory/work caps refuse unresolved inputs cooperatively; there is no hard preemption of a single large integer multiplication.

The small reference lives in `scripts/exact_commit/semantic_json.py` and is separate from production decoders. Focused offline tests use an independent JSON-tree executor and enumerate token products and integer sampling decisions, including aliases, fixed positions and zero mass. `formal/MWPC/SemanticProfiles.lean` verifies distinct finite ternary witnesses, the 3^m rectangle-cover obstruction, and the lower bound directly for natural-coefficient bilinear plans with a positive common tensor scale. Clearing rational denominators, the real-coefficient support reduction, signed optimality, sampling law and source implementation remain written proofs or tested obligations; the whole software is not Lean-verified.

The frozen MDLM/JsonLogic integration capture was produced from clean commit `2be61a9`. Its rational inputs, config, original results and checksums are archived in `docs/artifacts/raw/m36_semantic_reference_v1/`. An actual pinned CPU MDLM prediction fills five lexical slots; support domains were declared before execution and independently of target labels. All 108 represented rules agree with the official pinned JsonLogic consumer on eight Boolean records. Two positive targets yield sixteen samples each; the declared parity target has zero mass on this canvas. Full local logits passed an independent softmax audit; those logits are not part of the Git archive. Offline replay recomputes the published rational inputs against independent enumeration and recorded masses, and is now a focused CI check. It does not load a model or repeat the optional consumer/full-logit audits.

Capture commands and optional audits are in `REPRODUCING.md`. This illustration is not a held-out generalization benchmark. Human review remains required before promoting the supplement to the main scientific claim.

7 Complete adaptive and core audit

The continuation was frozen in clean producing commit 30e6ac3, before running `configs/experiments/m36_adaptive_semantics_v1.json`. It uses the unchanged capture above, enumerates every label vector, and shares eager and enumeration preparation across targets. Raw configuration, preparation costs, all outcomes, support/provenance metadata and checksums are in `docs/artifacts/raw/m36_adaptive_semantics_v1/`. The table is generated from that archive; no favorable subset or new benchmark was selected.

Method	Positive	Zero	Refused	Batch ms
Adaptive	51	717	0	5.343
Certified core	51	717	0	1.009
Eager profiles	51	717	0	2.050
Enumeration	51	717	0	0.007
Prefix control	51	717	0	0.611

Table 1: All 256 label vectors, 3 seeds: 768 rows and 108 original-token programs. 17 labels have positive mass. Batch time is the median over positive rows, each requesting 4 samples; shared preparation is additional.

Independent enumeration checks every target mass, every returned token path and equivalence of each structural core against every original path, including paths of zero model weight. Current-code CI recomputes the first positive and first zero target under both adaptive and certified-core inference, including deterministic seeded samples. No method returned an incorrect classification or a resource refusal. The maximum adaptive rejected-rule count was 7, within the eight-record bound.

The strongest small-support control is enumeration. The core’s shorter median batch does not offset its preparation here: certification summed over targets costs 2.532941 seconds, and evaluating their archived weights costs 0.232866 seconds. Shared all-profile preparation is 0.002862 seconds; enumeration/oracle preparation is 0.003114 seconds. Per-target core preparation is charged once, not three times for three seeds. Core certificates may be amortized over later compatible predictions, but this short audit does not measure that trajectory. The prefix control uses CARS’s trie subtraction with a perfect finite oracle; it does not time native CARS or EPIC. These results expose overheads; Theorems R1–R3, with their stated comparator and reuse conditions, supply the mathematical guarantees independently of these timings.

References

- [1] Andreas Björklund, Thore Husfeldt, Petteri Kaski, and Mikko Koivisto. Fourier meets möbius: fast subset convolution, 2006.
- [2] Susmit Jha, Sumit Gulwani, Sanjit A. Seshia, and Ashish Tiwari. Oracle-guided component-based program synthesis. In *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering*, pages 215–224, 2010.
- [3] Paweł Parys, Sairam Vaidya, Taylor Berg-Kirkpatrick, and Loris D’Antoni. Constrained adaptive rejection sampling. *arXiv preprint arXiv:2510.01902*, 2025.
- [4] Jianchang Su and Wei Zhang. Constraints are graphs, not chains: Exact decoding for diffusion language models. *arXiv preprint arXiv:2609.32900v1*, 2026.
- [5] Xinyu Wang, Isil Dillig, and Rishabh Singh. Synthesis of data completion scripts using finite tree automata, 2017.